設備控制 API 的規範

基本資訊

項目	內容
實作檔案	api/大概率是 API 吧/control_device/control_device.py
方法	POST / CGI stdin
Endpoint	/control_device/control_device.py
Content-Type	application/json; charset=utf-8
控制模式	CONTROL_MODE=mock 或 CONTROL_MODE=mqtt
App 互動	App 傳入 user 控制封包或 guest_token 控制封包。
ESP 互動	mqtt 模式下 API publish MQTT 命令至 ESP32，ESP32 以 status 封包回報。

POST Request Parameters

欄位	型別	Required	說明
auth_type	string	true	user 或 guest_token。
user_id	string	auth_type=user 時 true	屋主或家人成員 ID。
guest_token	string	auth_type=guest_token 時 true	GUEST_ 開頭明文令牌；GT_ token_id 必須拒絕。
family_id	integer	true	目前操作場域 ID。
device_id	string	true	目標設備 ID。
action	string	true	UNLOCK、LOCK、ON、OFF、OPEN、CLOSE 等。
parameters	object	false	控制參數，例如 duration。

App → API Request Packet

```
{
  "payload": {
    "auth_type": "user",
    "user_id": "admin_001",
    "family_id": 12,
    "device_id": "ESP32_LOCK_001",
    "action": "UNLOCK",
    "parameters": {
      "duration": 3
    }
  }
}
```

```
{
  "payload": {
    "auth_type": "guest_token",
    "guest_token": "GUEST_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
    "family_id": 12,
    "device_id": "ESP32_LOCK_001",
    "action": "UNLOCK",
    "parameters": {
      "duration": 3
    }
  }
}
```

Processing Rules

編號	規則
1	讀取 POST JSON payload，確認 payload 為 object。
2	依 auth_type 分流：user 以 user_families 驗證角色；guest_token 檢查 GUEST_ 格式並以 SHA-256 查 token_hash。
3	驗證 family_id、device_id、設備狀態、配對狀態與可執行 action。
4	建立 command_id，產生 command_payload。
5	寫入 control_commands，保存 request_payload 與初始 status。
6	mock 模式直接產生 response_payload，寫入 device_telemetry 並將 control_commands.status 更新為 SUCCEEDED。
7	mqtt 模式 publish 到 home/{family_id}/device/{device_id}/cmd，status 更新為 PUBLISHED。

8	所有成功與拒絕結果皆寫入 audit_logs。

API 對 SQL 寫入內容

資料表	寫入 / 更新欄位	觸發時機與說明
control_commands	INSERT command_id, family_id, device_id, actor_id, actor_type, action, parameters, control_mode, target_topic, request_payload, status, reason, created_at, published_at, completed_at	每次控制請求必定寫入。拒絕時 status=DENIED；mock 成功時 status=SUCCEEDED；mqtt 發送後 status=PUBLISHED。
guest_tokens	UPDATE used_count = used_count + 1, last_used_at = NOW()	僅 auth_type=guest_token 且令牌驗證成功時更新。
device_telemetry	INSERT family_id, device_id, command_id, status, physical_state, telemetry_data, battery, rssi	mock 模式立即寫入；mqtt 模式由 ESP32 status 回報後寫入。
audit_logs	INSERT command_id, user_id, guest_token_id, actor_type, device_id, family_id, action, parameters, raw_data, status, decision, reason, timestamp, prev_hash, current_hash	控制允許、控制拒絕、token 格式錯誤、角色拒絕都要寫入稽核。

API → ESP32 MQTT Command Packet

項目	內容
Topic	home/{family_id}/device/{device_id}/cmd
QoS	建議 1
Retain	false
觸發條件	CONTROL_MODE=mqtt 且驗證通過

```
{
  "command_id": "CMD_20260707151855_067a1154",
  "family_id": 12,
  "device_id": "ESP32_LOCK_001",
  "action": "UNLOCK",
  "parameters": {
    "duration": 3
  },
  "actor_id": "admin_001",
  "actor_type": "USER",
  "timestamp": "2026-07-07 15:18:55",
  "nonce": "hex_nonce"
}
```

ESP32 → API Status Packet

項目	內容
Topic	home/{family_id}/device/{device_id}/status
處理者	mqtt_status_worker.py 或 device_status_update.py
保存位置	control_commands.response_payload、device_telemetry.telemetry_data、audit_logs.raw_data

```
{
  "command_id": "CMD_20260707151855_067a1154",
  "family_id": 12,
  "device_id": "ESP32_LOCK_001",
  "status": "SUCCEEDED",
  "physical_state": "UNLOCKED",
  "battery": 87,
  "rssi": -52,
  "error_code": null
}
```

API → App Success Response

```
{
  "status": "Success",
  "message": "MOCK_CONTROL_SUCCEEDED",
  "data": {
```

```
    "command_id": "CMD_...",
    "family_id": 12,
    "device_id": "ESP32_LOCK_001",
    "action": "UNLOCK",
    "control_mode": "mock",
    "command_status": "SUCCEEDED",
    "target_topic": null,
    "response_payload": {
      "command_id": "CMD_20260707151855_067a1154",
      "family_id": 12,
      "device_id": "ESP32_LOCK_001",
      "status": "SUCCEEDED",
      "physical_state": "UNLOCKED",
      "battery": 87,
      "rssi": -52,
      "error_code": null
    }
  }
}
```

Error Responses

HTTP 狀態	錯誤碼	說明
400	INVALID_JSON / MISSING_FIELD	JSON 格式錯誤或缺少必要欄位。
403	ROLE_DENIED / TOKEN_FORMAT_INVALID / TOKEN_EXPIRED / TOKEN_USED_UP	角色不符、訪客令牌格式錯誤、過期或次數耗盡。
404	DEVICE_NOT_FOUND / FAMILY_NOT_FOUND / TOKEN_NOT_FOUND	查無設備、場域或令牌。
409	DEVICE_REVOKED / DUPLICATE_RECORD	設備狀態衝突或不可重複操作。
500	DB_DRIVER_MISSING / INTERNAL_ERROR	資料庫驅動、SQL 或伺服器內部錯誤。

注意事項

- App 端不可把 GT_ token_id 當 guest_token 傳入；正式版只接受 GUEST_ 明文令牌。
- mock 模式用於 API/MySQL 流程測試，不會真的控制 ESP32。
- mqtt 模式下 App 不應假設設備已完成動作；需等 ESP32 status 回報或透過儀表板查詢最新狀態。
- 失敗請求仍會寫入 audit_logs，方便追蹤越權或錯誤 token。

App 呼叫範例

App 端以 HTTPS POST 送出 JSON；本機測試可用 curl 或 printf 對應 CGI stdin。

```
POST /control_device/control_device.py
Content-Type: application/json; charset=utf-8

{
  "payload": {
    "auth_type": "user",
    "user_id": "admin_001",
    "family_id": 12,
    "device_id": "ESP32_LOCK_001",
    "action": "UNLOCK",
    "parameters": {
      "duration": 3
    }
  }
}
```

```
curl -X POST "http://localhost:8000/control_device/control_device.py" \
  -H "Content-Type: application/json" \
  -d '{"payload": {"auth_type": "user", "user_id": "admin_001", "family_id": 12, "device_id": "ESP32_LOCK_001", "action":
"UNLOCK", "parameters": {"duration": 3}}}'
```

本機測試範例

```
export CONTROL_MODE=mock
export DB_HOST=localhost
```

```
export DB_USER=vboxuser
export DB_PASSWORD='12345678'
export DB_NAME=devicemanagement

printf '%s'
'{"payload":{"auth_type":"user","user_id":"admin_001","family_id":12,"device_id":"ESP32_LOCK_001","action":"UNLOCK","parameters"
:{"duration":3}}}' \
| python -u api/大概率是API吧/control_device/control_device.py 2>&1
```